Link Prediction on the LastFM Asia Social Network

Emery Chuttarsing
emeryc.chuttarsing@berkeley.edu

Chenyu Tang
tcy@berkeley.edu

Abstract

We study link prediction on the LastFM Asia social network (7,624 users, 27,806 links) using two approaches. First, we formulate the task as regularized low-rank Matrix Factorization and optimize it with full-batch gradient descent under a negative-sampling train/test protocol, achieving a test AUC of 0.8435. Second, we implement an encoder–decoder graph learning model with a GCN encoder and an MLP decoder, achieving a test AUC 0.9579. Both models shows strong ability to predict the links in social network.

1 Introduction

Social networks act as the backbone of modern digital interaction, connecting users based on shared interests, geography, or real-world relationships. A critical challenge for platforms like LastFM is **Link Prediction**: the ability to recommend new connections to users (e.g., "People you may know"). Accurately predicting these missing links drives user engagement and retention.

In this project, we analyze the **LastFM Asia Social Network** dataset[1]. This graph contains 7,624 nodes (users) and 27,806 edges (mutual follower relationships). The graph is relatively sparse, with a density of approximately 0.09%, making the task of inferring latent relationships a significant computational challenge.

Our goal is to uncover the latent structure of this network to predict the likelihood of friendship between any pair of users. We approach this objective from two complementary perspectives:

- **Optimization baseline:** We formulate link prediction as a low-rank **Matrix Factorization** problem and solve it using first-order methods (Batch GD and SGD).
- **Machine learning method:** We implement an **encoder–decoder** link prediction model, where a graph encoder (e.g., a GCN[2]) produces node embeddings and a neural decoder scores candidate edges.

This provides both an interpretable low-rank baseline and a more expressive learning-based approach.

2 Mathematical Formulation for Optimization

2.1 Matrix Factorization

We model the link prediction task on an undirected graph $G = (V, E)$ where V is the set of N users and E represents the observed edges. We define the adjacency matrix $A \in \{0,1\}^{N\times N}$ such that:

$$A_{ij} = \begin{cases} 1 & \text{if users } i \text{ and } j \text{ are connected,} \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

We assume that the existence of an edge depends on the similarity between the users' latent features. We aim to learn a low-rank embedding matrix $U \in \mathbb{R}^{N\times k}$, where each row $u_i \in \mathbb{R}^k$ is a k-dimensional vector representing user i. The parameter k is a hyperparameter chosen such that $k \ll N$ (e.g., $k = 10$), forcing the model to compress information.

The predicted score for a link between i and j is given by the inner product:

$$\hat{A}_{ij} = u_i^T u_j. \tag{2}$$

2.2 The Optimization Problem

We seek to find the matrix U that minimizes the reconstruction error between the predicted scores and the actual adjacency matrix. We formulate this as an unconstrained optimization problem minimizing the **Regularized Squared Loss**:

$$\min_{U\in\mathbb{R}^{N\times k}} f(U) = \sum_{(i,j)\in\Omega} (A_{ij} - u_i^T u_j)^2 + \lambda \sum_{i=1}^{N} \|u_i\|_2^2, \tag{3}$$

where Ω is the set of training pairs (observed edges and sampled non-edges), the first term measures

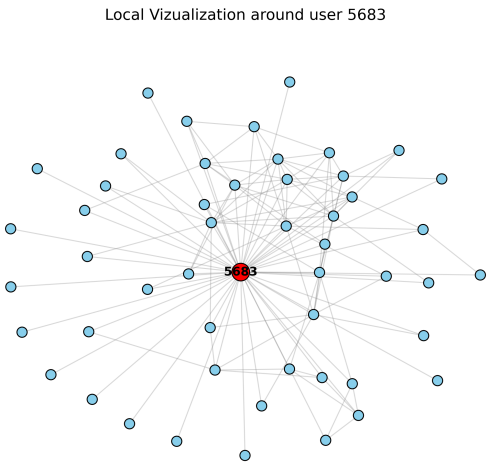


Figure 1: Visualization of a sub-region of the LastFM Asia social graph (Ego-network of the highest degree node).

the **fidelity** of the reconstruction (Least Squares), and the second term is the L_2 **regularization** (Ridge), controlled by λ, which penalizes large vector norms to reduce overfitting and improve numerical stability.

2.3 Problem Properties

The objective function $f(U)$ is a fourth-order polynomial with respect to the entries of U, making it **non-convex**. However, it is **bi-convex**: if we fix the vector u_i, the problem becomes convex with respect to u_j, and vice-versa. This property suggests that iterative first-order optimization methods, such as Gradient Descent, are suitable candidates for finding a high-quality local minimum.

3 Methodology

3.1 Data Pre-processing and Negative Sampling

The raw LastFM dataset consists of "implicit feedback": we only observe existing friendships ($A_{ij} = 1$), but we do not have explicit data on non-friendships ($A_{ij} = 0$).

Training a model solely on positive examples would lead to a trivial degenerate solution where the model predicts 1 for every pair. To address this, we employ **Negative Sampling**:

- We treat all unobserved links as potential non-friendships.
- For every positive edge (u, v) in the training set, we randomly sample a user k such that (u, k) does not exist in the graph.
- We assign a target label of 0 to these sampled pairs.

This balances the classes and forces the model to learn a decision boundary between friends and non-friends.

Train-Test Split: We hide 20% of the positive edges to serve as the ground truth for evaluation (A_{test}). The remaining 80% of positive edges, combined with the generated negative samples, form the training set (Ω_{train}).

3.2 Optimization Method: Full-Batch Gradient Descent

We chose **Full-Batch Gradient Descent (BGD)** as our optimization baseline. Unlike stochastic methods that approximate the gradient using subsets of data, BGD computes the exact gradient over the entire dataset Ω_{train} at every iteration.

3.2.1 Derivation

The gradient of the objective function $f(U)$ with respect to a user vector u_i is derived as follows:

$$\nabla_{u_i} f(U) = -2 \sum_{j \in \mathcal{N}(i)} (A_{ij} - u_i^T u_j) u_j + 2\lambda u_i \tag{4}$$

where $\mathcal{N}(i)$ represents the set of users interacting with i (both positive and sampled negative neighbors).

3.2.2 Algorithm Implementation

The iterative procedure is defined as:

1. **Initialization:** Initialize U with small random values drawn from a normal distribution $\mathcal{N}(0, 0.1)$.
2. **Gradient Computation:** For each epoch, calculate the error vector $E = A_{\text{train}} - UU^T$ and compute the full gradient matrix $\nabla f(U)$.
3. **Update Rule:** Update all user embeddings simultaneously:

$$U^{(t+1)} \leftarrow U^{(t)} - \alpha \nabla f(U^{(t)}) \tag{5}$$

where α is the learning rate.

3.3 Machine Learning Method: Encoder–Decoder Link Prediction

While Matrix Factorization provides an optimization-based baseline, it models link likelihood using only a bilinear similarity function. To increase model expressiveness, we implement a machine learning approach based on an **encoder–decoder** architecture commonly used in link prediction.

3.3.1 Model Architecture

Our ML method follows:

$$Z = f_\theta(X, E_{\text{mp}}), \qquad s_{ij} = g_\phi(z_i, z_j), \tag{6}$$

where:

- f_θ is a graph encoder producing node embeddings $Z \in \mathbb{R}^{N \times d}$,
- g_ϕ is a decoder producing an edge score (logit) $s_{ij} \in \mathbb{R}$ for a candidate pair (i, j).

3.3.2 Encoder: Graph Convolutional Network (GCN)

We adopt a Graph Convolutional Network (GCN) encoder[2]. Starting from node features $H^{(0)} = X$, each layer performs neighborhood aggregation:

$$H^{(\ell+1)} = \sigma\Big(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(\ell)}W^{(\ell)}\Big), \tag{7}$$

where $\tilde{A}$ is the adjacency matrix with self-loops, $\tilde{D}$ is its degree matrix, $W^{(\ell)}$ are trainable weights, and $\sigma(\cdot)$ is a nonlinearity. The final node embeddings are $Z = H^{(L)}$.

3.3.3 Decoder: MLP Edge Scoring

To score an edge (i, j), we use a multi-layer perceptron (MLP) decoder that operates on pairwise features derived from (z_i, z_j). We use the composite feature vector:

$$\phi(z_i, z_j) = [z_i \parallel z_j \parallel z_i \odot z_j \parallel |z_i - z_j|], \tag{8}$$

where $\odot$ denotes elementwise product and $\parallel$ denotes concatenation. The decoder then computes:

$$s_{ij} = \text{MLP}_\phi(\phi(z_i, z_j)), \tag{9}$$

where s_{ij} is a logit (unbounded real value).

3.3.4 Training Objective

We train the model using binary cross-entropy with logits:

$$\mathcal{L} = -\frac{1}{|\Omega|}\sum_{(i,j)\in\Omega}\Big[y_{ij}\log\sigma(s_{ij}) + (1 - y_{ij})\log\big(1 - \sigma(s_{ij})\big)\Big]. \tag{10}$$

where Ω is the set of labeled training pairs, $y_{ij} \in \{0, 1\}$, and $\sigma(\cdot)$ is the sigmoid function. We additionally use weight decay and dropout for regularization.

3.4 Evaluation Metrics

Since the link prediction problem is essentially a binary classification task on imbalanced data, simple accuracy is insufficient. We evaluate performance using the **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)**[3]. The AUC measures the probability that a randomly chosen missing link (true friend) is ranked higher by the model than a randomly chosen nonexistent link. An AUC of 0.5 indicates random guessing, while 1.0 indicates perfect prediction.

4 Experiments

4.1 Full-Batch Gradient Descent algorithm

4.1.1 Experimental Setup

The model was trained with an embedding dimension of $k = 10$. We used a learning rate of $\alpha = 0.01$ and a regularization parameter $\lambda = 0.01$ to prevent overfitting. The optimization was run for 50 epochs, at which point the loss function converged to a stable minimum.

4.1.2 Quantitative Performance

Table 1 summarizes the performance metrics on the test set. The primary metric for success is the AUC-ROC, as it is threshold-independent. For Precision, Recall, and F1-Score, we applied a sigmoid function to the raw scores and used a standard classification threshold of 0.5.

Metric	Score
AUC-ROC	**0.8435**
Recall	0.9087
F1-Score	0.7271
Precision	0.6061

Table 1: Performance metrics of the Batch Gradient Descent model on the held-out test set.

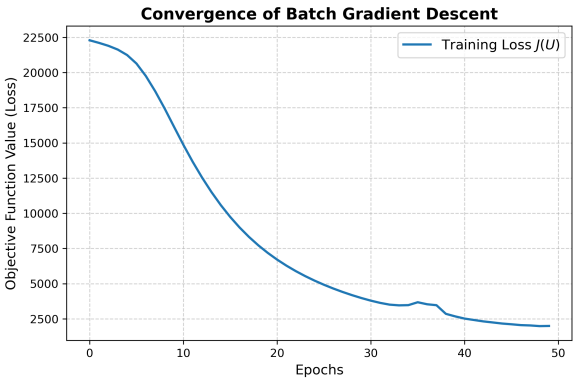


Figure 2: Training Loss vs. Epochs. The monotonic decrease indicates stable convergence towards a local minimum.

4.1.3 Convergence Analysis

Figure 2 (below) illustrates the trajectory of the objective function $f(U)$ for baseline model during training. The smooth, monotonic decrease of the loss confirms the stability of the Batch Gradient Descent method and the correctness of the gradient derivation.

4.2 Encoder–Decoder Link Prediction

4.2.1 Experimental Setup

The best-performing ML model found in the sweep uses a **GCN encoder** with an **MLP decoder**. Based on validation average precision, the best-performing configuration employs a three-layer GCN encoder with a hidden dimension of 256 and an embedding dimension of 256. Link scores are computed using an MLP decoder that operates on concatenated pairwise features, including the node embeddings, their element-wise product, and their absolute difference. Model parameters are optimized using the Adam optimizer with a learning rate 1×10^{-3} and a weight decay of 5×10^{-5}. To mitigate overfitting, a dropout rate of 0.2 is applied during training.

4.2.2 Quantitative Performance

Table 2 reports test performance for the selected model. The test set contains $n = 5560$ labeled pairs with a positive rate of 0.5 (due to 1:1 negative sampling).

Metric	Score
AUC-ROC	**0.9579**
Accuracy	0.9074
Average Precision (AP)	0.9655
BCE Loss	0.3303

Table 2: Test metrics for the best ML model (GCN encoder + MLP decoder), selected by validation AP.

4.3 Discussion of Results

The high AUC vaalue indicates that both models has successfully learned the social structure. It implies that if we pick a random pair of real friends and a random pair of non-friends, the model will correctly rank the real friends higher 84%/95% of the time.

The GCN+MLP model substantially improves link prediction performance relative to the baseline. We attribute this improvement to two factors:

1. **Neighborhood-aware representations:** the GCN encoder aggregates information from local graph structure, which is critical in social networks with strong community structure.

2. **Nonlinear edge scoring:** the MLP decoder can learn interactions beyond simple similarity, improving separability between true edges and sampled non-edges.

References

[1] Benedek Rozemberczki and Rik Sarkar. Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages —, Virtual Event, Ireland, 2020. ACM.

[2] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[3] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

Appendix

A Code for Data Preprocess

```
import pandas as pd
import numpy as np
import random
from sklearn.model_selection import train_test_split

df = pd.read_csv('lastfm_asia_edges.csv')

# 2. Remapping of IDs

unique_nodes = pd.unique(df[['node_1', 'node_2']].values.ravel('K'))
num_nodes = len(unique_nodes)
id_map = {old_id: new_id for new_id, old_id in enumerate(unique_nodes)}

df['u'] = df['node_1'].map(id_map)
df['v'] = df['node_2'].map(id_map)

# List of all positive link
pos_edges = df[['u', 'v']].values

# 3. Split Train / Test (Hidding 20% of the link for final exam)
train_pos, test_pos = train_test_split(pos_edges, test_size=0.2, random_state=42)

# 4. Negative link generation

def generate_negatives(pos_edges, num_nodes):
    neg_edges = []
    pos_set = set(map(tuple, pos_edges))

    for u, v in pos_edges:

        while True:
            k = np.random.randint(0, num_nodes)
            if k != u and (u, k) not in pos_set and (k, u) not in pos_set:
                neg_edges.append([u, k])
                break
    return np.array(neg_edges)

# generative negative for train set
train_neg = generate_negatives(train_pos, num_nodes)
```

B Code for Full-Bactch Gradient Descent Algorithm

```
def batch_gradient_descent(num_nodes, train_pos, train_neg, k=10, lr=0.01,
    lambda_reg=0.01, epochs=50):
    """
    Implementation de Batch Gradient Descent for Matrix Factorization
    """
    # Random initialisation of embedding (Matrix U)
    U = np.random.normal(0, 0.1, (num_nodes, k))

    loss_history = []

    X = np.vstack([train_pos, train_neg])
    y = np.concatenate([np.ones(len(train_pos)), np.zeros(len(train_neg))])

    for epoch in range(epochs):

        users = X[:, 0]
        items = X[:, 1]

        u_vecs = U[users] # Shape (M, k)
        v_vecs = U[items] # Shape (M, k)

        preds = np.sum(u_vecs * v_vecs, axis=1)
```

```
        # 2. Error calculation
        errors = preds - y

        # Loss calculation
        loss = np.sum(errors ** 2) + lambda_reg * np.sum(U ** 2)
        loss_history.append(loss)

        # 3. Backward Pass : Gradient calculus

        grad_U = np.zeros_like(U)

        grad_u_part = 2 * errors[:, np.newaxis] * v_vecs
        grad_v_part = 2 * errors[:, np.newaxis] * u_vecs

        np.add.at(grad_U, users, grad_u_part)
        np.add.at(grad_U, items, grad_v_part)

        #regularization term
        grad_U += 2 * lambda_reg * U

        # 4. Update Step
        U = U - lr * grad_U

        if epoch % 5 == 0:
            print(f"Epoch {epoch}/{epochs} - Loss: {loss:.4f}")

    return U, loss_history

print("Start Batch Gradient Descent...")
U_optimized, history = batch_gradient_descent(num_nodes, train_pos, train_neg)
```

C Code for Encoder-Decoder Link Prediction

C.1 Encoder

```
"""Encoder modules for link prediction.

Encoders map node features (and optionally graph structure) to node embeddings
`z = f(x, edge_index)` with a consistent interface.

This module requires PyTorch. Some encoders optionally depend on
`torch_geometric`; if it is not installed, attempting to instantiate a PyG-based
encoder raises an informative ImportError.
"""

from __future__ import annotations

from typing import Callable

import torch

__all__ = ["MLPEncoder", "NodeEmbeddingEncoder", "GCNEncoder", "GraphSAGEEncoder"]


def _get_activation(name: str) -> torch.nn.Module:
    """Map an activation name to a torch.nn.Module."""

    key = name.strip().lower()
    if key == "relu":
        return torch.nn.ReLU()
    if key == "gelu":
        return torch.nn.GELU()
    if key == "tanh":
        return torch.nn.Tanh()
    if key in {"identity", "linear", "none"}:
        return torch.nn.Identity()
    raise ValueError(
        f"Unknown activation='{name}'. Supported: relu, gelu, tanh, identity."
```

```
    )


def _is_integer_dtype(t: torch.Tensor) -> bool:
    if t.is_complex():
        return False
    if t.dtype == torch.bool:
        return False
    return not t.is_floating_point()


def _validate_x(x: torch.Tensor, *, in_channels: int | None = None) -> None:
    if not isinstance(x, torch.Tensor):
        raise ValueError(f"x must be a torch Tensor; got {type(x).__name__}.")
    if x.dim() != 2:
        raise ValueError(f"x must have shape [N, F]; got shape {tuple(x.shape)}.")
    if x.is_complex():
        raise ValueError(f"x must be real-valued; got dtype={x.dtype}.")
    if in_channels is not None and int(x.size(-1)) != int(in_channels):
        raise ValueError(
            "x feature dimension must match in_channels; "
            f"got x.shape[1]={int(x.size(-1))}, in_channels={int(in_channels)}."
        )


def _validate_edge_index(edge_index: torch.Tensor) -> None:
    if not isinstance(edge_index, torch.Tensor):
        raise ValueError(
            f"edge_index must be a torch Tensor; got {type(edge_index).__name__}."
        )
    if edge_index.dim() != 2 or edge_index.size(0) != 2:
        raise ValueError(
            f"edge_index must have shape [2, E]; got shape {tuple(edge_index.shape)
                }."
        )
    if not _is_integer_dtype(edge_index):
        raise ValueError(
            "edge_index must have an integer dtype convertible to torch.long; "
            f"got dtype={edge_index.dtype}."
        )


def _require_torch_geometric(*, reason: str) -> Callable[..., object]:
    """Import torch_geometric and return its `nn` module or raise with a hint."""

    try:
        import torch_geometric.nn as pyg_nn  # type: ignore[import-not-found]
    except ImportError as exc:  # pragma: no cover
        raise ImportError(
            f"torch_geometric is required for {reason}. "
            "Install with `pip install torch-geometric` (see "
            "https://pytorch-geometric.readthedocs.io/)."
        ) from exc
    return pyg_nn


class MLPEncoder(torch.nn.Module):
    """A simple MLP encoder that ignores graph structure."""

    def __init__(
        self,
        in_channels: int,
        hidden_channels: int,
        out_channels: int,
        num_layers: int = 2,
        dropout: float = 0.0,
        activation: str = "relu",
        use_batchnorm: bool = False,
    ) -> None:
        super().__init__()
```

```
        if in_channels <= 0:
            raise ValueError(f"in_channels must be > 0; got {in_channels}.")
        if hidden_channels <= 0:
            raise ValueError(f"hidden_channels must be > 0; got {hidden_channels}.")
        if out_channels <= 0:
            raise ValueError(f"out_channels must be > 0; got {out_channels}.")
        if num_layers <= 0:
            raise ValueError(f"num_layers must be >= 1; got {num_layers}.")
        if not (0.0 <= float(dropout) < 1.0):
            raise ValueError(f"dropout must be in [0.0, 1.0); got {dropout}.")

        self.in_channels = int(in_channels)
        self.out_channels = int(out_channels)
        self.num_layers = int(num_layers)
        self.use_batchnorm = bool(use_batchnorm)

        self._activation = _get_activation(activation)
        self._dropout = torch.nn.Dropout(p=float(dropout))

        layers: list[torch.nn.Linear] = []
        for layer_idx in range(self.num_layers):
            in_dim = self.in_channels if layer_idx == 0 else int(hidden_channels)
            out_dim = (
                self.out_channels
                if layer_idx == self.num_layers - 1
                else int(hidden_channels)
            )
            layers.append(torch.nn.Linear(in_dim, out_dim))
        self.lins = torch.nn.ModuleList(layers)

        if self.use_batchnorm and self.num_layers > 1:
            self.batch_norms = torch.nn.ModuleList(
                [
                    torch.nn.BatchNorm1d(int(hidden_channels))
                    for _ in range(self.num_layers - 1)
                ]
            )
        else:
            self.batch_norms = torch.nn.ModuleList()

    def forward(
        self, x: torch.Tensor, edge_index: torch.Tensor | None = None
    ) -> torch.Tensor:
        """Compute node embeddings from node features.

        Args:
            x: Node feature tensor with shape [N, F].
            edge_index: Ignored (present for API consistency).

        Returns:
            Node embedding tensor with shape [N, out_channels].
        """

        _validate_x(x, in_channels=self.in_channels)

        h = x
        for i, lin in enumerate(self.lins):
            h = lin(h)
            is_last = i == (len(self.lins) - 1)
            if not is_last:
                if self.use_batchnorm:
                    h = self.batch_norms[i](h)
                h = self._activation(h)
                h = self._dropout(h)
        return h

    def encode(
        self, x: torch.Tensor, edge_index: torch.Tensor | None = None
    ) -> torch.Tensor:
```

```
        """Alias for :meth:`forward`."""

        return self.forward(x, edge_index=edge_index)


class NodeEmbeddingEncoder(torch.nn.Module):
    """Learnable node embeddings (no message passing)."""

    def __init__(self, num_nodes: int, out_channels: int) -> None:
        super().__init__()

        if num_nodes <= 0:
            raise ValueError(f"num_nodes must be > 0; got {num_nodes}.")
        if out_channels <= 0:
            raise ValueError(f"out_channels must be > 0; got {out_channels}.")

        self.num_nodes = int(num_nodes)
        self.out_channels = int(out_channels)
        self.embedding = torch.nn.Embedding(self.num_nodes, self.out_channels)

    def forward(
        self, x: torch.Tensor | None = None, edge_index: torch.Tensor | None = None
    ) -> torch.Tensor:
        """Return learned embeddings for all nodes.

        Args:
            x: Optional node feature tensor. If provided, its first dimension
                must match `num_nodes`. Feature values are ignored.
            edge_index: Ignored (present for API consistency).

        Returns:
            Node embedding tensor with shape [num_nodes, out_channels].
        """

        if x is not None:
            if not isinstance(x, torch.Tensor):
                raise ValueError(
                    "x must be a torch Tensor or None; "
                    f"got {type(x).__name__}."
                )
            if x.dim() < 1:
                raise ValueError(
                    "x must have at least 1 dimension; "
                    f"got shape {tuple(x.shape)}."
                )
            if int(x.size(0)) != self.num_nodes:
                raise ValueError(
                    "x.shape[0] must match num_nodes; "
                    f"got x.shape[0]={int(x.size(0))}, num_nodes={self.num_nodes}."
                )
        return self.embedding.weight

    def encode(
        self, x: torch.Tensor | None = None, edge_index: torch.Tensor | None = None
    ) -> torch.Tensor:
        """Alias for :meth:`forward`."""

        return self.forward(x=x, edge_index=edge_index)


class GCNEncoder(torch.nn.Module):
    """Graph Convolutional Network (GCN) encoder (requires torch_geometric)."""

    def __init__(
        self,
        in_channels: int,
        hidden_channels: int,
        out_channels: int,
        num_layers: int = 2,
        dropout: float = 0.0,
```

```
        activation: str = "relu",
        use_batchnorm: bool = False,
        add_self_loops: bool = True,
        normalize: bool = True,
    ) -> None:
        super().__init__()

        pyg_nn = _require_torch_geometric(reason="GCNEncoder")
        GCNConv = pyg_nn.GCNConv

        if in_channels <= 0:
            raise ValueError(f"in_channels must be > 0; got {in_channels}.")
        if hidden_channels <= 0:
            raise ValueError(f"hidden_channels must be > 0; got {hidden_channels}.")
        if out_channels <= 0:
            raise ValueError(f"out_channels must be > 0; got {out_channels}.")
        if num_layers <= 0:
            raise ValueError(f"num_layers must be >= 1; got {num_layers}.")
        if not (0.0 <= float(dropout) < 1.0):
            raise ValueError(f"dropout must be in [0.0, 1.0); got {dropout}.")

        self.in_channels = int(in_channels)
        self.out_channels = int(out_channels)
        self.num_layers = int(num_layers)
        self.use_batchnorm = bool(use_batchnorm)

        self._activation = _get_activation(activation)
        self._dropout = torch.nn.Dropout(p=float(dropout))

        convs: list[torch.nn.Module] = []
        for layer_idx in range(self.num_layers):
            in_dim = self.in_channels if layer_idx == 0 else int(hidden_channels)
            out_dim = (
                self.out_channels
                if layer_idx == self.num_layers - 1
                else int(hidden_channels)
            )
            convs.append(
                GCNConv(
                    in_channels=in_dim,
                    out_channels=out_dim,
                    add_self_loops=bool(add_self_loops),
                    normalize=bool(normalize),
                )
            )
        self.convs = torch.nn.ModuleList(convs)

        if self.use_batchnorm and self.num_layers > 1:
            self.batch_norms = torch.nn.ModuleList(
                [
                    torch.nn.BatchNorm1d(int(hidden_channels))
                    for _ in range(self.num_layers - 1)
                ]
            )
        else:
            self.batch_norms = torch.nn.ModuleList()

    def forward(self, x: torch.Tensor, edge_index: torch.Tensor | None) -> torch.
        Tensor:
        """Compute node embeddings using message passing over `edge_index`."""

        if edge_index is None:
            raise ValueError("edge_index is required for GCNEncoder; got None.")
        _validate_x(x, in_channels=self.in_channels)
        _validate_edge_index(edge_index)

        ei = edge_index
        if ei.dtype != torch.long:
            ei = ei.to(dtype=torch.long)
        if ei.device != x.device:
```

```
            ei = ei.to(device=x.device)

        h = x
        for i, conv in enumerate(self.convs):
            h = conv(h, ei)
            is_last = i == (len(self.convs) - 1)
            if not is_last:
                if self.use_batchnorm:
                    h = self.batch_norms[i](h)
                h = self._activation(h)
                h = self._dropout(h)
        return h

    def encode(self, x: torch.Tensor, edge_index: torch.Tensor | None) -> torch.
        Tensor:
        """Alias for :meth:`forward`."""

        return self.forward(x, edge_index)


class GraphSAGEEncoder(torch.nn.Module):
    """GraphSAGE encoder (requires torch_geometric)."""

    def __init__(
        self,
        in_channels: int,
        hidden_channels: int,
        out_channels: int,
        num_layers: int = 2,
        dropout: float = 0.0,
        activation: str = "relu",
        use_batchnorm: bool = False,
        aggr: str = "mean",
    ) -> None:
        super().__init__()

        pyg_nn = _require_torch_geometric(reason="GraphSAGEEncoder")
        SAGEConv = pyg_nn.SAGEConv

        if in_channels <= 0:
            raise ValueError(f"in_channels must be > 0; got {in_channels}.")
        if hidden_channels <= 0:
            raise ValueError(f"hidden_channels must be > 0; got {hidden_channels}.")
        if out_channels <= 0:
            raise ValueError(f"out_channels must be > 0; got {out_channels}.")
        if num_layers <= 0:
            raise ValueError(f"num_layers must be >= 1; got {num_layers}.")
        if not (0.0 <= float(dropout) < 1.0):
            raise ValueError(f"dropout must be in [0.0, 1.0); got {dropout}.")

        self.in_channels = int(in_channels)
        self.out_channels = int(out_channels)
        self.num_layers = int(num_layers)
        self.use_batchnorm = bool(use_batchnorm)
        self.aggr = str(aggr)

        self._activation = _get_activation(activation)
        self._dropout = torch.nn.Dropout(p=float(dropout))

        convs: list[torch.nn.Module] = []
        for layer_idx in range(self.num_layers):
            in_dim = self.in_channels if layer_idx == 0 else int(hidden_channels)
            out_dim = (
                self.out_channels
                if layer_idx == self.num_layers - 1
                else int(hidden_channels)
            )
            convs.append(SAGEConv(in_dim, out_dim, aggr=self.aggr))
        self.convs = torch.nn.ModuleList(convs)

```

```
        if self.use_batchnorm and self.num_layers > 1:
            self.batch_norms = torch.nn.ModuleList(
                [
                    torch.nn.BatchNorm1d(int(hidden_channels))
                    for _ in range(self.num_layers - 1)
                ]
            )
        else:
            self.batch_norms = torch.nn.ModuleList()

    def forward(self, x: torch.Tensor, edge_index: torch.Tensor | None) -> torch.
        Tensor:
        """Compute node embeddings using GraphSAGE message passing."""

        if edge_index is None:
            raise ValueError("edge_index is required for GraphSAGEEncoder; got None.
                ")
        _validate_x(x, in_channels=self.in_channels)
        _validate_edge_index(edge_index)

        ei = edge_index
        if ei.dtype != torch.long:
            ei = ei.to(dtype=torch.long)
        if ei.device != x.device:
            ei = ei.to(device=x.device)

        h = x
        for i, conv in enumerate(self.convs):
            h = conv(h, ei)
            is_last = i == (len(self.convs) - 1)
            if not is_last:
                if self.use_batchnorm:
                    h = self.batch_norms[i](h)
                h = self._activation(h)
                h = self._dropout(h)
        return h

    def encode(self, x: torch.Tensor, edge_index: torch.Tensor | None) -> torch.
        Tensor:
        """Alias for :meth:`forward`."""

        return self.forward(x, edge_index)
```

C.2 Decoder

```
"""Decoder modules for link prediction.

Decoders map node embeddings `z` and a set of edges `edge_label_index` to edge
logits.

This module is torch-only.
"""

from __future__ import annotations

import torch

__all__ = ["DotProductDecoder", "MLPDecoder"]


def _get_activation(name: str) -> torch.nn.Module:
    key = name.strip().lower()
    if key == "relu":
        return torch.nn.ReLU()
    if key == "gelu":
        return torch.nn.GELU()
    if key == "tanh":
        return torch.nn.Tanh()
    if key in {"identity", "linear", "none"}:
```

```
        return torch.nn.Identity()
    raise ValueError(
        f"Unknown activation='{name}'. Supported: relu, gelu, tanh, identity."
    )


def _is_integer_dtype(t: torch.Tensor) -> bool:
    if t.is_complex():
        return False
    if t.dtype == torch.bool:
        return False
    return not t.is_floating_point()


def _validate_z(z: torch.Tensor) -> None:
    if not isinstance(z, torch.Tensor):
        raise ValueError(f"z must be a torch Tensor; got {type(z).__name__}.")
    if z.dim() != 2:
        raise ValueError(f"z must have shape [N, D]; got shape {tuple(z.shape)}.")
    if z.is_complex():
        raise ValueError(f"z must be real-valued; got dtype={z.dtype}.")
    if not z.is_floating_point():
        raise ValueError(f"z must be a floating-point tensor; got dtype={z.dtype}.")


def _validate_edge_label_index(edge_label_index: torch.Tensor) -> None:
    if not isinstance(edge_label_index, torch.Tensor):
        raise ValueError(
            "edge_label_index must be a torch Tensor; "
            f"got {type(edge_label_index).__name__}."
        )
    if edge_label_index.dim() != 2 or edge_label_index.size(0) != 2:
        raise ValueError(
            "edge_label_index must have shape [2, M]; "
            f"got shape {tuple(edge_label_index.shape)}."
        )
    if not _is_integer_dtype(edge_label_index):
        raise ValueError(
            "edge_label_index must have an integer dtype convertible to torch.long;
                "
            f"got dtype={edge_label_index.dtype}."
        )


def _standardize_edge_index(
    z: torch.Tensor, edge_label_index: torch.Tensor
) -> torch.Tensor:
    ei = edge_label_index
    if ei.device != z.device:
        ei = ei.to(device=z.device)
    if ei.dtype != torch.long:
        ei = ei.to(dtype=torch.long)
    return ei


class DotProductDecoder(torch.nn.Module):
    """Dot-product edge decoder."""

    def forward(self, z: torch.Tensor, edge_label_index: torch.Tensor) -> torch.
        Tensor:
        """Compute edge logits via dot product.

        Args:
            z: Node embeddings with shape [N, D].
            edge_label_index: Edge endpoints with shape [2, M] (src, dst).

        Returns:
            Logits tensor with shape [M].
        """

```

```
        _validate_z(z)
        _validate_edge_label_index(edge_label_index)

        ei = _standardize_edge_index(z, edge_label_index)
        src = ei[0]
        dst = ei[1]

        z_src = z.index_select(0, src)
        z_dst = z.index_select(0, dst)
        logits = (z_src * z_dst).sum(dim=-1)

        if logits.dim() != 1 or int(logits.numel()) != int(ei.size(1)):
            raise ValueError(
                "DotProductDecoder must return a 1D tensor of length M; "
                f"got shape {tuple(logits.shape)} for M={int(ei.size(1))}."
            )
        return logits


class MLPDecoder(torch.nn.Module):
    """MLP-based edge decoder over pairs of node embeddings."""

    _SUPPORTED_PAIR_FEATURES = {"concat", "hadamard", "abs_diff", "
        concat_hadamard_abs"}

    def __init__(
        self,
        in_channels: int,
        hidden_channels: int,
        num_layers: int = 2,
        dropout: float = 0.0,
        activation: str = "relu",
        pair_feature: str = "concat_hadamard_abs",
    ) -> None:
        super().__init__()

        if in_channels <= 0:
            raise ValueError(f"in_channels must be > 0; got {in_channels}.")
        if hidden_channels <= 0:
            raise ValueError(f"hidden_channels must be > 0; got {hidden_channels}.")
        if num_layers <= 0:
            raise ValueError(f"num_layers must be >= 1; got {num_layers}.")
        if not (0.0 <= float(dropout) < 1.0):
            raise ValueError(f"dropout must be in [0.0, 1.0); got {dropout}.")

        pf = pair_feature.strip().lower()
        if pf not in self._SUPPORTED_PAIR_FEATURES:
            raise ValueError(
                "pair_feature must be one of: "
                + ", ".join(sorted(self._SUPPORTED_PAIR_FEATURES))
                + f"; got '{pair_feature}'."
            )

        self.in_channels = int(in_channels)
        self.hidden_channels = int(hidden_channels)
        self.num_layers = int(num_layers)
        self.pair_feature = pf

        self._activation = _get_activation(activation)
        self._dropout = torch.nn.Dropout(p=float(dropout))

        input_dim = self._pair_feature_dim(self.in_channels, self.pair_feature)

        layers: list[torch.nn.Linear] = []
        for layer_idx in range(self.num_layers):
            if self.num_layers == 1:
                in_dim = input_dim
                out_dim = 1
            else:
                in_dim = input_dim if layer_idx == 0 else self.hidden_channels
```

```
                out_dim = (
                    1 if layer_idx == self.num_layers - 1 else self.hidden_channels
                )
            layers.append(torch.nn.Linear(int(in_dim), int(out_dim)))
        self.lins = torch.nn.ModuleList(layers)

    @staticmethod
    def _pair_feature_dim(in_channels: int, pair_feature: str) -> int:
        if in_channels <= 0:
            raise ValueError(f"in_channels must be > 0; got {in_channels}.")
        pf = pair_feature.strip().lower()
        if pf not in MLPDecoder._SUPPORTED_PAIR_FEATURES:
            raise ValueError(
                "pair_feature must be one of: "
                + ", ".join(sorted(MLPDecoder._SUPPORTED_PAIR_FEATURES))
                + f"; got '{pair_feature}'."
            )
        return {
            "concat": 2 * in_channels,
            "hadamard": in_channels,
            "abs_diff": in_channels,
            "concat_hadamard_abs": 4 * in_channels,
        }[pf]

    def _pair_features(self, z_u: torch.Tensor, z_v: torch.Tensor) -> torch.Tensor:
        if self.pair_feature == "concat":
            return torch.cat([z_u, z_v], dim=-1)
        if self.pair_feature == "hadamard":
            return z_u * z_v
        if self.pair_feature == "abs_diff":
            return (z_u - z_v).abs()
        if self.pair_feature == "concat_hadamard_abs":
            return torch.cat([z_u, z_v, z_u * z_v, (z_u - z_v).abs()], dim=-1)
        raise RuntimeError(f"Unsupported pair_feature='{self.pair_feature}'.")

    def forward(self, z: torch.Tensor, edge_label_index: torch.Tensor) -> torch.
        Tensor:
        """Compute edge logits using an MLP over pairwise features.

        Args:
            z: Node embeddings with shape [N, D].
            edge_label_index: Edge endpoints with shape [2, M] (src, dst).

        Returns:
            Logits tensor with shape [M].
        """

        _validate_z(z)
        _validate_edge_label_index(edge_label_index)

        if int(z.size(1)) != self.in_channels:
            raise ValueError(
                "z feature dimension must match in_channels; "
                f"got z.shape[1]={int(z.size(1))}, in_channels={self.in_channels}."
            )

        ei = _standardize_edge_index(z, edge_label_index)
        src = ei[0]
        dst = ei[1]

        z_u = z.index_select(0, src)
        z_v = z.index_select(0, dst)
        h = self._pair_features(z_u, z_v)

        for i, lin in enumerate(self.lins):
            h = lin(h)
            is_last = i == (len(self.lins) - 1)
            if not is_last:
                h = self._activation(h)
                h = self._dropout(h)
```

```
        logits = h.view(-1)
        m = int(ei.size(1))
        if int(logits.numel()) != m:
            raise ValueError(
                "MLPDecoder output must have length M; "
                f"expected M={m}, got {int(logits.numel())}."
            )
        return logits
```

C.3 Trainer

```
# ===== src/trainer.py =====
"""Training loops for link prediction.

This module provides a minimal, torch-only trainer that supports two modes:

1) Full-batch mode:
   Uses split-like objects exposing `edge_label_index` and `edge_label`, and
   relies on :func:`src.train_utils.train_step_fullbatch` /
   :func:`src.train_utils.evaluate_fullbatch`.

2) Edge-batch mode:
   Uses torch DataLoaders yielding `EdgeBatch` samples and trains/evaluates over
   those batches.

The trainer is model-agnostic: the model must be able to produce node
embeddings from `(x, edge_index)` with a flexible signature (handled by
`src.train_utils.get_node_embeddings`). A decoder can be provided explicitly,
derived from `model.decoder` when present, or left as `None` to default to a
dot-product decoder downstream.
"""

from __future__ import annotations

import math
from dataclasses import dataclass
from typing import Any, Callable, cast

try:
    import torch
except ImportError as e:  # pragma: no cover
    raise ImportError("PyTorch is required. Install with `pip install torch`.") from
        e

from torch.utils.data import DataLoader

__all__ = [
    "TrainerConfig",
    "FitResult",
    "Trainer",
    "flatten_metrics",
    "select_decoder",
    "train_one_epoch_edgebatch",
    "evaluate_edgebatch",
    "fit",
]

Decoder = Callable[[torch.Tensor, torch.Tensor], torch.Tensor]


@dataclass(slots=True)
class TrainerConfig:
    """Configuration for :func:`fit` and :class:`Trainer`."""

    max_epochs: int = 200
    eval_every: int = 1
    device: str = "cpu"
    chunk_size: int | None = None
```

```
    grad_clip: float | None = None
    pos_weight: float | None = None

    early_stopping: bool = True
    monitor: str = "ap"  # one of: "ap", "auc", "loss"
    mode: str | None = None  # if None, infer from monitor
    patience: int = 20
    min_delta: float = 0.0
    restore_best: bool = True

    verbose: bool = True
    log_every: int = 1

    def __post_init__(self) -> None:
        if self.max_epochs <= 0:
            raise ValueError(f"max_epochs must be > 0; got {self.max_epochs}.")
        if self.eval_every <= 0:
            raise ValueError(f"eval_every must be > 0; got {self.eval_every}.")
        if self.log_every <= 0:
            raise ValueError(f"log_every must be > 0; got {self.log_every}.")
        if self.patience < 0:
            raise ValueError(f"patience must be >= 0; got {self.patience}.")
        if not math.isfinite(float(self.min_delta)) or float(self.min_delta) < 0.0:
            raise ValueError(
                f"min_delta must be a finite non-negative float; got {self.min_delta
                    }."
            )
        if self.grad_clip is not None:
            grad_clip = float(self.grad_clip)
            if not math.isfinite(grad_clip) or grad_clip <= 0.0:
                raise ValueError(
                    f"grad_clip must be a finite positive float; got {self.grad_clip
                        }."
                )
        if self.pos_weight is not None:
            pos_weight = float(self.pos_weight)
            if not math.isfinite(pos_weight) or pos_weight <= 0.0:
                raise ValueError(
                    "pos_weight must be a finite positive float; "
                    f"got {self.pos_weight}."
                )
        if self.mode is not None and self.mode not in {"max", "min"}:
            raise ValueError(f'mode must be "max" or "min"; got "{self.mode}".')


@dataclass(slots=True)
class FitResult:
    """Output from :func:`fit`."""

    history: list[dict[str, float]]
    best_epoch: int | None
    best_value: float | None
    test_metrics: dict[str, float] | None


def flatten_metrics(metrics: dict[str, float], prefix: str) -> dict[str, float]:
    """Flatten a metrics dict with a prefix."""

    return {f"{prefix}{k}": float(v) for k, v in metrics.items()}


def select_decoder(model: torch.nn.Module, decoder: Decoder | None) -> Decoder |
    None:
    """Select a decoder callable.

    Selection order:
      1) Explicit `decoder` argument, when provided.
      2) `model.decoder` attribute, when present and callable.
      3) `None` (callers should default to a dot-product decoder).
    """
```

```
    if decoder is not None:
        return decoder
    model_decoder = getattr(model, "decoder", None)
    if callable(model_decoder):
        return cast(Decoder, model_decoder)
    return None


def _to_device_tensor(
    t: torch.Tensor | None, device: torch.device
) -> torch.Tensor | None:
    """Move a tensor to `device` (best-effort conversion)."""

    if t is None:
        return None
    if not isinstance(t, torch.Tensor):
        t = torch.as_tensor(t)
    return t.to(device=device)


def _require_attr(obj: object, attr: str, *, name: str) -> Any:
    if not hasattr(obj, attr):
        raise ValueError(f"{name} is missing required attribute '{attr}'.")
    return getattr(obj, attr)


def _infer_early_stopping_mode(monitor: str) -> str:
    key = monitor.strip().lower()
    if key in {"ap", "auc"}:
        return "max"
    if key == "loss":
        return "min"
    raise ValueError('monitor must be one of: "ap", "auc", "loss".')


def _format_log_line(
    epoch: int,
    max_epochs: int,
    train_metrics: dict[str, float],
    val_metrics: dict[str, float] | None,
    *,
    monitor: str,
) -> str:
    pieces = [f"epoch={epoch}/{max_epochs}"]
    if "loss" in train_metrics:
        pieces.append(f"train_loss={train_metrics['loss']:.4f}")
    if monitor in train_metrics:
        pieces.append(f"train_{monitor}={train_metrics[monitor]:.4f}")
    if val_metrics is not None:
        if "loss" in val_metrics:
            pieces.append(f"val_loss={val_metrics['loss']:.4f}")
        if monitor in val_metrics:
            pieces.append(f"val_{monitor}={val_metrics[monitor]:.4f}")
    return " ".join(pieces)


def _extract_batch_tensors(batch: object) -> tuple[torch.Tensor, torch.Tensor]:
    edge_label_index = _require_attr(batch, "edge_label_index", name="batch")
    edge_label = _require_attr(batch, "edge_label", name="batch")

    if not isinstance(edge_label_index, torch.Tensor):
        edge_label_index = torch.as_tensor(edge_label_index)
    if not isinstance(edge_label, torch.Tensor):
        edge_label = torch.as_tensor(edge_label)

    return edge_label_index, edge_label


def train_one_epoch_edgebatch(
```

```
    model: torch.nn.Module,
    data: object,
    loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    *,
    device: str | torch.device = "cpu",
    decoder: Decoder | None = None,
    grad_clip: float | None = None,
    pos_weight: float | None = None,
    chunk_size: int | None = None,
) -> dict[str, float]:
    """Train for one epoch over an edge-label DataLoader.

    Notes:
        For simplicity, this implementation recomputes node embeddings `z` for
        every batch (i.e., each batch is a new forward pass over all nodes).
        This is correct but may be slower than more specialized strategies.
    """

    from .train_utils import (
        bce_with_logits_loss,
        compute_binary_metrics,
        get_node_embeddings,
        logits_from_edgebatch,
        predict_logits,
    )

    device_t = torch.device(device)
    model.to(device_t)
    model.train()

    if grad_clip is not None:
        if not math.isfinite(float(grad_clip)) or float(grad_clip) <= 0.0:
            raise ValueError(
                f"grad_clip must be a finite positive float; got {grad_clip}."
            )

    x = _to_device_tensor(_require_attr(data, "x", name="data"), device_t)
    if x is None:
        raise ValueError("data.x must be provided for edge-batch training.")
    edge_index_raw = getattr(data, "edge_index", None)
    edge_index = _to_device_tensor(edge_index_raw, device_t)
    if edge_index is not None:
        edge_index = edge_index.to(dtype=torch.long)

    logits_cpu: list[torch.Tensor] = []
    labels_cpu: list[torch.Tensor] = []
    total_edges = 0
    loss_sum = 0.0

    for batch in loader:
        edge_label_index, edge_label = _extract_batch_tensors(batch)

        edge_label_index_t = _to_device_tensor(edge_label_index, device_t)
        if edge_label_index_t is None:
            raise ValueError("batch.edge_label_index must be provided.")
        edge_label_index_t = edge_label_index_t.to(dtype=torch.long)

        labels_t = _to_device_tensor(edge_label, device_t)
        if labels_t is None:
            raise ValueError("batch.edge_label must be provided.")
        labels_t = labels_t.to(dtype=torch.float32).view(-1)

        try:
            optimizer.zero_grad(set_to_none=True)
        except TypeError:  # pragma: no cover
            optimizer.zero_grad()

        z = get_node_embeddings(model, x, edge_index)

```

```
        if chunk_size is None:
            logits = logits_from_edgebatch(z, batch, decoder=decoder)
        else:
            logits = predict_logits(
                z, edge_label_index_t, decoder=decoder, chunk_size=chunk_size
            )

        loss = bce_with_logits_loss(
            logits, labels_t, pos_weight=pos_weight, reduction="mean"
        )
        loss.backward()

        if grad_clip is not None:
            torch.nn.utils.clip_grad_norm_(model.parameters(), float(grad_clip))

        optimizer.step()

        batch_size = int(labels_t.numel())
        total_edges += batch_size
        loss_sum += float(loss.detach().item()) * batch_size

        logits_cpu.append(logits.detach().to(device="cpu").view(-1))
        labels_cpu.append(labels_t.detach().to(device="cpu").view(-1))

    if total_edges <= 0:
        raise ValueError("Edge-batch loader produced no samples.")

    all_logits = torch.cat(logits_cpu, dim=0)
    all_labels = torch.cat(labels_cpu, dim=0)
    metrics = compute_binary_metrics(all_labels, all_logits)
    metrics["loss"] = float(loss_sum / float(total_edges))
    return metrics


@torch.no_grad()
def evaluate_edgebatch(
    model: torch.nn.Module,
    data: object,
    loader: DataLoader,
    *,
    device: str | torch.device = "cpu",
    decoder: Decoder | None = None,
    chunk_size: int | None = None,
    pos_weight: float | None = None,
) -> dict[str, float]:
    """Evaluate a model over an edge-label DataLoader."""

    from .train_utils import (
        bce_with_logits_loss,
        compute_binary_metrics,
        get_node_embeddings,
        logits_from_edgebatch,
        predict_logits,
    )

    device_t = torch.device(device)

    x = _to_device_tensor(_require_attr(data, "x", name="data"), device_t)
    if x is None:
        raise ValueError("data.x must be provided for edge-batch evaluation.")
    edge_index_raw = getattr(data, "edge_index", None)
    edge_index = _to_device_tensor(edge_index_raw, device_t)
    if edge_index is not None:
        edge_index = edge_index.to(dtype=torch.long)

    prev_mode = model.training
    model.to(device_t)
    model.eval()
    try:
        z = get_node_embeddings(model, x, edge_index)
```

```
        logits_cpu: list[torch.Tensor] = []
        labels_cpu: list[torch.Tensor] = []
        total_edges = 0
        loss_sum = 0.0

        for batch in loader:
            edge_label_index, edge_label = _extract_batch_tensors(batch)

            edge_label_index_t = _to_device_tensor(edge_label_index, device_t)
            if edge_label_index_t is None:
                raise ValueError("batch.edge_label_index must be provided.")
            edge_label_index_t = edge_label_index_t.to(dtype=torch.long)

            labels_t = _to_device_tensor(edge_label, device_t)
            if labels_t is None:
                raise ValueError("batch.edge_label must be provided.")
            labels_t = labels_t.to(dtype=torch.float32).view(-1)

            if chunk_size is None:
                logits = logits_from_edgebatch(z, batch, decoder=decoder)
            else:
                logits = predict_logits(
                    z, edge_label_index_t, decoder=decoder, chunk_size=chunk_size
                )

            loss = bce_with_logits_loss(
                logits, labels_t, pos_weight=pos_weight, reduction="mean"
            )

            batch_size = int(labels_t.numel())
            total_edges += batch_size
            loss_sum += float(loss.detach().item()) * batch_size

            logits_cpu.append(logits.detach().to(device="cpu").view(-1))
            labels_cpu.append(labels_t.detach().to(device="cpu").view(-1))

        if total_edges <= 0:
            raise ValueError("Edge-batch loader produced no samples.")

        all_logits = torch.cat(logits_cpu, dim=0)
        all_labels = torch.cat(labels_cpu, dim=0)
        metrics = compute_binary_metrics(all_labels, all_logits)
        metrics["loss"] = float(loss_sum / float(total_edges))
        return metrics
    finally:
        if prev_mode:
            model.train()


def fit(
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    *,
    cfg: TrainerConfig,
    train_data: object,
    val_data: object | None = None,
    test_data: object | None = None,
    train_loader: DataLoader | None = None,
    val_loader: DataLoader | None = None,
    test_loader: DataLoader | None = None,
    decoder: Decoder | None = None,
) -> FitResult:
    """Main training loop for link prediction."""

    from .train_utils import EarlyStopping, evaluate_fullbatch, train_step_fullbatch

    decoder_fn = select_decoder(model, decoder)

    if val_loader is not None and val_data is None:
```

```
        raise ValueError("val_loader was provided but val_data is None.")
    if test_loader is not None and test_data is None:
        raise ValueError("test_loader was provided but test_data is None.")

    monitor = cfg.monitor.strip().lower()
    if monitor not in {"ap", "auc", "loss"}:
        raise ValueError('cfg.monitor must be one of: "ap", "auc", "loss".')

    device_t = torch.device(cfg.device)

    use_val = val_data is not None
    use_test = test_data is not None

    early_stopper: EarlyStopping | None = None
    best_epoch: int | None = None

    if cfg.early_stopping and use_val:
        mode = cfg.mode if cfg.mode is not None else _infer_early_stopping_mode(
            monitor)
        early_stopper = EarlyStopping(
            patience=cfg.patience, mode=mode, min_delta=cfg.min_delta
        )

    history: list[dict[str, float]] = []

    for epoch in range(1, cfg.max_epochs + 1):
        if train_loader is not None:
            train_metrics = train_one_epoch_edgebatch(
                model,
                train_data,
                train_loader,
                optimizer,
                device=device_t,
                decoder=decoder_fn,
                grad_clip=cfg.grad_clip,
                pos_weight=cfg.pos_weight,
                chunk_size=cfg.chunk_size,
            )
        else:
            train_metrics = train_step_fullbatch(
                model,
                train_data,
                optimizer,
                device=device_t,
                decoder=decoder_fn,
                chunk_size=cfg.chunk_size,
                grad_clip=cfg.grad_clip,
                pos_weight=cfg.pos_weight,
            )

        val_metrics: dict[str, float] | None = None
        should_eval = use_val and (epoch % cfg.eval_every == 0)
        should_stop = False

        if should_eval and val_data is not None:
            if val_loader is not None:
                val_metrics = evaluate_edgebatch(
                    model,
                    val_data,
                    val_loader,
                    device=device_t,
                    decoder=decoder_fn,
                    chunk_size=cfg.chunk_size,
                    pos_weight=cfg.pos_weight,
                )
            else:
                val_metrics = evaluate_fullbatch(
                    model,
                    val_data,
                    device=device_t,
```

```
                    decoder=decoder_fn,
                    chunk_size=cfg.chunk_size,
                )

            if early_stopper is not None:
                if monitor not in val_metrics:
                    raise ValueError(
                        f"cfg.monitor='{cfg.monitor}' not found in validation
                            metrics: "
                        f"{sorted(val_metrics.keys())}"
                    )
                current = float(val_metrics[monitor])
                prev_best = float(early_stopper.best_value)
                should_stop = early_stopper.step(current, model)
                if (
                    float(early_stopper.best_value) != prev_best
                    and math.isfinite(current)
                ):
                    best_epoch = int(epoch)

        record: dict[str, float] = {"epoch": float(epoch)}
        record.update(flatten_metrics(train_metrics, "train_"))
        if val_metrics is not None:
            record.update(flatten_metrics(val_metrics, "val_"))
        history.append(record)

        if cfg.verbose and (epoch % cfg.log_every == 0):
            print(
                _format_log_line(
                    epoch,
                    cfg.max_epochs,
                    train_metrics,
                    val_metrics,
                    monitor=monitor,
                )
            )

        if should_stop:
            if cfg.verbose:
                print(
                    f"Early stopping at epoch={epoch} "
                    f"(best_epoch={best_epoch}, "
                    f"best_{monitor}={early_stopper.best_value:.6g})."
                )
            break

    if (
        cfg.restore_best
        and early_stopper is not None
        and early_stopper.best_state_dict is not None
    ):
        model.load_state_dict(early_stopper.best_state_dict)

    test_metrics: dict[str, float] | None = None
    if use_test and test_data is not None:
        if test_loader is not None:
            test_metrics = evaluate_edgebatch(
                model,
                test_data,
                test_loader,
                device=device_t,
                decoder=decoder_fn,
                chunk_size=cfg.chunk_size,
                pos_weight=cfg.pos_weight,
            )
        else:
            test_metrics = evaluate_fullbatch(
                model,
                test_data,
                device=device_t,
```

```
                decoder=decoder_fn,
                chunk_size=cfg.chunk_size,
            )

    best_value: float | None = None
    if best_epoch is not None and early_stopper is not None:
        best_value = float(early_stopper.best_value)

    return FitResult(
        history=history,
        best_epoch=best_epoch,
        best_value=best_value,
        test_metrics=test_metrics,
    )


class Trainer:
    """OO wrapper around the module-level :func:`fit`."""

    def __init__(
        self,
        model: torch.nn.Module,
        optimizer: torch.optim.Optimizer,
        cfg: TrainerConfig,
        decoder: Decoder | None = None,
    ) -> None:
        self.model = model
        self.optimizer = optimizer
        self.cfg = cfg
        self.decoder = decoder

    def fit(
        self,
        *,
        train_data: object,
        val_data: object | None = None,
        test_data: object | None = None,
        train_loader: DataLoader | None = None,
        val_loader: DataLoader | None = None,
        test_loader: DataLoader | None = None,
    ) -> FitResult:
        """Fit using the stored model/optimizer/cfg."""

        return fit(
            self.model,
            self.optimizer,
            cfg=self.cfg,
            train_data=train_data,
            val_data=val_data,
            test_data=test_data,
            train_loader=train_loader,
            val_loader=val_loader,
            test_loader=test_loader,
            decoder=self.decoder,
        )
```